A Replicable Theory-Guided LLM Workflow for Measuring Legal Delegation and Administrative Discretion

Anonymous Authors

Abstract. Coding complex legal texts for nuanced institutional concepts is difficult to scale, replicate, and audit. Large language models can reduce the cost of coding and text analysis. However, one-shot classifications often obscure the judgments underlying the final result and are difficult to reproduce across models and runs. We develop a replicable, theory-guided LLM workflow that converts expert coding rules into a versioned, auditable measurement instrument. The workflow decomposes complex legal concepts into ordered judgments. It preserves prompts, settings, evidence, and intermediate outputs while distinguishing generative classifications from predefined coding rules. We demonstrate the approach by measuring legislative delegation and administrative discretion in U.S. financial-regulation laws. The workflow first predicts whether a law delegates authority. It then classifies the amount of authority and the strength of statutory constraints before assigning a five-category ordinal discretion rank. We implement the workflow in a research prototype that displays prompts, settings, outputs, validation rules, and execution records. We evaluate binary delegation classification and three rank-classification strategies across multiple language models on the 169-law expert-coded benchmark. Delegation classification performs well despite severe class imbalance, while discretion-rank agreement remains modest and varies across model-strategy combinations.

Keywords: legal natural language processing · large language models · legislative delegation · administrative discretion · reproducible workflows · quantitative legal coding

1 Introduction

Researchers studying political institutions routinely convert complex legal texts into structured measures of authority, constraint, and institutional design [7]. Manually coding laws is difficult because institutional concepts are latent, multidimensional, and span multiple statutory provisions. This requires expert coding rules, structured judgments, and preserving the evidence supporting each decision. Administrative discretion is a useful example because it depends both on the authority delegated by the legislature and the constraints that govern its use [5,10].

Large language models offer a scalable approach to legal coding. However, one-shot classification collapses these linked judgments into a single output. A

model may assign the correct rank for the wrong reason, produce a rank inconsistent with its stated assessment of delegation or constraints, or make errors that cannot be traced to a specific coding decision. Single-prompt approaches are therefore difficult to audit, revise, and reproduce across models and runs. A replicable measurement system must instead decompose the construct into ordered judgments, preserve the evidence and intermediate outputs, and apply the coding rules consistently.

We address this problem with a theory-guided workflow that turns the measurement model into an executable sequence of decisions. First, the workflow determines whether a law delegates qualifying authority. If it does, the system separately classifies the amount of authority and the strength of statutory constraints, then combines those judgments to assign a discretion rank. We implement the workflow in an anonymized research prototype to execute and record prompts, settings, intermediate outputs, and validation rules within a versioned, auditable framework. This paper makes four principal contributions:

1. **Replicable measurement instrument.** We develop an LLM-based instrument for measuring legal delegation and administrative discretion. The complete workflow, rather than an isolated prompt, is archived, versioned, and auditable.
2. **Theory-guided workflow.** We translate an established formal model of delegation and discretion into an executable measurement workflow. The model identifies the concepts and relationships that define the variables used in empirical tests.
3. **Transparent validation.** We separate generative LLM judgments from deterministic theoretical mappings and preserve the intermediate evidence needed to diagnose disagreements.
4. **Research-facing prototype.** We enable domain researchers to apply and inspect the measurement process without directly managing prompts, APIs, model orchestration, or structured-output parsing.

The objective is not to establish an autonomous legal coder, but rather to show how expert-defined measurement can be made scalable, inspectable, and auditable.

2 Measurement Task and Design Requirements

2.1 Delegation, Constraints, and Discretion

Delegation is the transfer of policymaking authority from a legislature to an executive or administrative actor, enabling the agent to modify policy or regulate conduct. Delegation alone, however, does not determine administrative discretion. Legislatures often impose substantive and procedural constraints, such as administrative standards, approval requirements, deadlines, reporting obligations, and oversight reviews, that restrict the agent's ability to set policy.

In principal-agent models of institutional design, administrative discretion is the agent's residual policy latitude after accounting for delegated authority and

the constraints on its use [5,4]. We operationalize this construct as

$$\text{Discretion} = D(1 - C), \tag{1}$$

where D represents delegated authority and C is the constraint index [5,10]. The workflow must therefore preserve the conceptual ordering in the measurement model: delegation is logically prior to discretion, and discretion depends jointly on the amount of authority transferred and the strength of constraints on its use.

This operationalization builds on cumulative research in legislative delegation: early formal models framed discretion as policy latitude balancing expertise against bureaucratic drift [4], empirical studies combined delegated authority and statutory constraints into quantitative indices for U.S. financial laws [10,13,8], and subsequent work explored algorithmic extraction from legal text [11,12,14].

The expert coding procedure operationalized these concepts directly from summaries of the law. Expert coders separately measured delegated authority and statutory constraints as continuous components, which were combined to produce the discretion index in Eq. 1. Separately, coders assigned each law to one of five ordinal discretion categories, 0–4, based on the total discretionary authority delegated in the legislative summary, taking into account both the authority granted and the restrictions on its exercise. The ordinal classification provides the expert-coded benchmark for the prediction task.

2.2 Ordinal operationalization

The five-category ordinal benchmark was coded separately from the continuous discretion index. Rank 0 denotes no delegation. Conditional on delegation, Ranks 1–4 correspond to the four combinations of delegated authority (low or high) and statutory constraints (strong or weak). The ordinal benchmark therefore provides a separate expert-coded classification of total discretionary authority.

Let x_i denote the legal text for law i. The measurement model contains three component variables: $d_i \in \{0, 1\}$ indicates whether the law delegates qualifying policymaking authority, $a_i \in \{\text{Low}, \text{High}\}$ denotes the amount of delegated authority, and $c_i \in \{\text{Strong}, \text{Weak}\}$ denotes the strength of statutory constraints. For the prediction workflow, the discretion rank is $y_i^{\text{map}} = m(d_i, a_i, c_i)$, where

$$m(d_i, a_i, c_i) = \begin{cases} 0, & d_i = 0, \\ 1, & d_i = 1,\ a_i = \text{Low},\ c_i = \text{Strong}, \\ 2, & d_i = 1,\ a_i = \text{Low},\ c_i = \text{Weak}, \\ 3, & d_i = 1,\ a_i = \text{High},\ c_i = \text{Strong}, \\ 4, & d_i = 1,\ a_i = \text{High},\ c_i = \text{Weak}. \end{cases} \tag{2}$$

This mapping defines the substantive meaning of the five discretion ranks. It is part of the measurement model rather than a learned relationship. This five-category ordering represents an expert-coded operationalization of the 2×2

authority-constraint space, rather than a continuous magnitude mechanically implied by $D(1-C)$. Substantively, the expert protocol reflects a lexicographic hierarchy in which the scope of delegated jurisdiction (D) is primary: an agency granted sweeping regulatory authority under tight procedural guardrails (Rank 3) exercises broader institutional policymaking impact than an agency with narrow, minor jurisdiction facing minimal statutory oversight (Rank 2). Ordinal evaluation metrics assess reproduction of this historical expert-coded ordering rather than estimation of a continuous magnitude.

2.3 Computational Requirements

LLMs have shown promise as low-cost annotators across text-annotation and qualitative tasks [6,3]. Those results do not establish reliability for specialized legal constructs. Prompted generative models may perform poorly when applying domain-expert codebooks to specialized historical or legal texts [16]. Legal NLP benchmarks such as LexGLUE and LegalBench likewise emphasize task-specific evaluation and expert-designed categories [1,9]. These findings motivate a conservative design in which LLM coding is evaluated against a controlled expert-coded benchmark and remains subject to expert review.

The workflow uses versioned LLM stages to estimate the component variables: $\hat{d}_i = g_D(x_i)$, $\hat{a}_i = g_A(x_i, \hat{d}_i)$, and $\hat{c}_i = g_C(x_i, \hat{d}_i, \hat{a}_i)$. The component stages also return structured evidence and rationales, denoted collectively by r_i. The prompts, inputs, outputs, evidence, and model settings are retained in the execution trace.

The system then applies one of three rank-classification strategies: Sequential Ordinal Cascade, Direct Multiclass Rank Prediction, and Two-Band Rank Classification. For strategy s, $\tilde{y}_i^{(s)} = h_s(x_i, \hat{d}_i, \hat{a}_i, \hat{c}_i, r_i)$. The theoretical mapping and the rank-classification strategies perform different functions. The mapping $m(d_i, a_i, c_i)$ defines the construct, whereas $h_s(\cdot)$ estimates the final rank under alternative computational strategies.

For each law, the system computes the theory-implied rank $y_i^{\text{map}} = m(\hat{d}_i, \hat{a}_i, \hat{c}_i)$ and compares it with the strategy-specific prediction. Define the consistency indicator

$$q_i^{(s)} = \mathbf{1}\left[\tilde{y}_i^{(s)} = y_i^{\text{map}}\right]. \tag{3}$$

A value of $q_i^{(s)} = 1$ indicates that the rank predicted by strategy s agrees with the rank implied by the model's own delegation, authority, and constraint classifications. A disagreement is retained and flagged for review rather than silently overwritten. This design allows the evaluation to distinguish benchmark agreement from internal theoretical consistency.

The workflow also draws on weak supervision and interactive human-feedback systems [15,2], as well as prompt-chaining systems that decompose LLM tasks into modular, controllable steps [18,17]. We do not claim to introduce human-in-the-loop annotation or prompt chaining. The contribution is to integrate these computational approaches with a substantive political-economy measurement

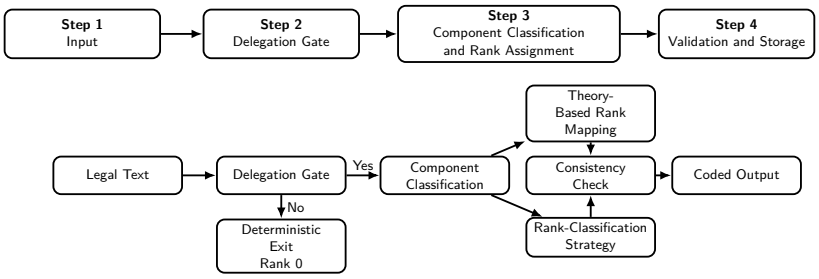


Fig. 1. Four-stage theory-guided classification workflow.

model so that expert coding rules determine the variables, dependencies, deterministic mappings, and validation checks used by the workflow.

3 Theory-Guided Workflow Architecture

The system was developed iteratively to align LLM judgments with the expert coding rules. The crucial methodological shift was from treating a single prompt as the coding instrument to treating the structured, multi-stage workflow graph as an instrument.

3.1 From Expert Coding Rules to Workflow Logic

We use the term *theory-guided AI* to describe a computational design in which substantive theory structures the inference process. Delegation is logically prior to discretion. The workflow first predicts whether the law delegates authority, $\hat{d}_i \in \{0, 1\}$. If $\hat{d}_i = 0$, it assigns Rank 0 deterministically and skips the component-classification and rank-prediction stages. If $\hat{d}_i = 1$, the workflow identifies the relevant administrative actors, classifies the amount of delegated authority, $\hat{a}_i \in \{\text{Low}, \text{High}\}$, and the strength of statutory constraints, $\hat{c}_i \in \{\text{Strong}, \text{Weak}\}$, and retains the supporting evidence and rationale, r_i. It then computes the component-implied rank, $y_i^{\text{map}} = m(\hat{d}_i, \hat{a}_i, \hat{c}_i)$, while the selected rank-classification strategy produces a separate prediction, $\tilde{y}_i^{(s)}$.

The workflow compares the two ranks using the consistency indicator $q_i^{(s)}$ and stores the component values, evidence, model settings, and execution trace. Figure 1 summarizes this theory-guided sequence.

This dependency structure addresses a recurrent failure of independent-column coding: a later variable, such as `discretion_rank`, cannot reliably use an earlier delegation decision unless the pipeline stores and forwards that decision. In the prototype, execution order is computed via topological sorting of the directed acyclic graph rather than assumed from the visual order of columns, preventing stale forward references. Each node records inputs, outputs, status, timing, and, where applicable, evidence and rationale.

3.2 Executable Workflow Operations

Table 1 translates the measurement logic into the executable sequence implemented by the workflow. It distinguishes LLM-assisted judgments from deter-

Table 1. Algorithmic execution of the theory-guided discretion workflow.

Step	Operation and Logic
1	**Load Source Text:** Read the legislative summary or statutory text x_i and attach stable identifiers.
2	**Delegation Gate (LLM):** Estimate $\hat{d}_i = g_D(x_i) \in \{0, 1\}$ and retain the supporting evidence and rationale.
3	**Early Exit (Deterministic):** If $\hat{d}_i = 0$, set the theory-implied rank $y_i^{\text{map}} = 0$, assign the required default values, and stop the delegated-discretion branch.
4	**Component Classification (LLM):** Conditional on $\hat{d}_i = 1$, identify the relevant administrative actors, classify delegated authority $\hat{a}_i \in \{\text{Low}, \text{High}\}$, classify statutory constraints $\hat{c}_i \in \{\text{Strong}, \text{Weak}\}$, and retain the supporting evidence and rationales r_i.
5	**Strategy-Specific Rank Prediction (LLM):** Apply the selected rank-classification strategy. The Sequential Ordinal Cascade uses successive early-exit binary decisions; Direct Multiclass predicts Ranks 1–4 in one step; and Two-Band first predicts a lower/higher rank band and then classifies within the selected band.
6	**Theoretical Consistency Validation (Deterministic):** Compute $y_i^{\text{map}} = m(\hat{d}_i, \hat{a}_i, \hat{c}_i)$ and compare it with the strategy-specific prediction $\tilde{y}_i^{(s)}$. Record the consistency indicator $q_i^{(s)}$ and flag any disagreement.
7	**Output Serialization:** Store the component classifications, theory-implied rank, strategy-specific rank, consistency flag, evidence, prompts, model settings, execution trace, runtimes, token use, cost, and final reported output.

ministic operations and shows how the system moves from source-text ingestion through delegation screening, component classification, rank prediction, consistency validation, and output storage.

The engine implements the workflow through document-input, LLM, conditional-routing, deterministic-value, validation, and output nodes. The workflow retains both the strategy-specific prediction $\tilde{y}_i^{(s)}$ and the theory-implied rank y_i^{map}. Strategy-specific predictions are used for comparative evaluation and are not silently replaced when they conflict with the deterministic component mapping. Instead, the validation node records the disagreement with an inconsistency flag. This separation allows researchers to evaluate both agreement with the expert-coded benchmark and consistency with the rules used to construct the discretion rank.

A one-shot prediction collapses the relationships among delegation, authority, constraints, and discretion into an opaque judgment. The workflow makes the measurement assumptions inspectable and testable because researchers can locate a disagreement at the Delegation Gate, authority classification, constraint classification, rank-classification strategy, or consistency check.

Figure 2 illustrates the reproducibility architecture. The system archives the source corpus, the workflow JSON schema and definition hash (SHA-256), model configurations, and a comprehensive node-level execution trace to ensure that any coded dataset can be audited and reconstructed. The prototype implements

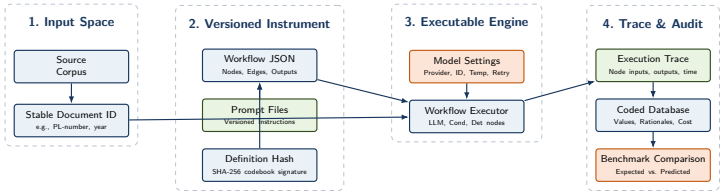


Fig. 2. Reproducibility and execution tracking pipeline.

the workflow as an auditable research instrument rather than as a production deployment system.

3.3 Audit Record and Reproducibility

Human expertise enters before, during, and after model execution. Before execution, domain experts define the construct, identify its constituent dimensions, delimit the source set, specify coding rules, and determine the relationships that the model must preserve. During evaluation, experts distinguish extraction errors from conceptual boundary cases. After evaluation, they decide whether a disagreement warrants a local case correction, a revision of the operational rule, or reconsideration of the historical benchmark.

Method-level revision. At the method level, researchers revise the codebook, source policy, stage prompts, dependencies, output definitions, and deterministic mappings. A method-level revision changes the general measurement rule and creates a new version to protect comparability across model runs.

Case-level adjudication. For an individual coded law, a researcher can inspect final values and node-by-node execution traces through an interactive visual dashboard. The interface displays side-by-side evidence rationales (r_i), flags discrepancies where $\tilde{y}_i^{(s)} \neq y_i^{\text{map}}$, and supports manual value overrides or AI re-evaluation with corrective feedback. Audit history tracks these local overrides without altering the global codebook or prompts for other cases.

4 Benchmark, Experimental Design, and Results

4.1 Benchmark and source discipline

The benchmark consists of 169 U.S. financial-regulation laws coded under a consistent protocol using legislative summaries from 1950–2013. We utilize CQ legislative summaries because they constitute the standardized, peer-reviewed corpus upon which the foundational political economy literature was constructed [5,10]. Using identical summary texts isolates LLM coding fidelity from the confounding effects of full-statute document length. Of these laws, 161 delegated authority and 8 contained no provisions delegating authority to executive agencies. Expert coders assigned 0–4 discretion labels directly from legislative summaries based on the total discretionary authority delegated and the restrictions on its exercise. We treat these labels as the benchmark for replication rather than as legal truth. Benchmark runs use the same summaries to preserve consistency with the original coding.

Table 2. Comparison of evaluation strategies.

Strategy	Inputs	Decision process	Output
One-shot baseline	Law text	Single LLM prediction with no intermediate guidance	Discretion Rank 0–4
Sequential Ordinal Cascade	Stage outputs	Evaluates early-exit binary splits: Rank 0 vs. Ranks 1–4 (delegation gate), then Rank 1 vs. Ranks 2–4, then Rank 2 vs. Ranks 3–4, and Rank 3 vs. Rank 4	Discretion Rank 0–4
Direct Multiclass Rank Prediction	Stage outputs	Predicts 1–4 directly using extracted authority and constraints	Discretion Rank 0–4
Two-Band Rank Classification	Stage outputs	Low-rank vs. high-rank band split, then within-band classification	Discretion Rank 0–4

4.2 Evaluation levels

The empirical evaluation asks whether LLMs can reproduce a theory-based measurement instrument, not merely whether they can generate plausible descriptions of statutes. The evaluation therefore occurs at several levels: whether the model identifies qualifying delegation, correctly characterizes the amount of authority, correctly characterizes the strength of constraints, produces a final rank consistent with those component judgments, and agrees with the historical benchmark.

We separate the evaluation into two distinct prediction tasks:

1. **Delegation Stage**: Binary delegation classification evaluated on the full available delegation benchmark ($N = 169$).
2. **Discretion Stage**: Ordinal rank classification (1–4) is evaluated only when a qualifying delegation exists.

4.3 Controlled prompt and workflow comparison

To evaluate discretion ranking, we implement and compare three staged computational strategies against a one-shot baseline. Table 2 summarizes these strategies.

4.4 Experimental protocol

We use the following experimental protocol:

– **Models**: We evaluate Gemini 3.5 (`gemini-3.5-flash-lite`), Qwen 3 (`qwen/qwen3-235b-a22b-2507`), DeepSeek (`deepseek/deepseek-chat`), Kimi K2.5 (`moonshotai/kimi-k2.5`), and GPT-OSS (`openai/gpt-oss-120b`). The unconstrained One-Shot Baseline is evaluated only on Gemini 3.5.
– **Settings**: We set temperature to 0.0 to reduce sampling variation and record exact model identifiers and execution timestamps.
– **Output Format**: Each node returns structured JSON validated against a Pydantic schema.

– **Retries and Failures**: If a model call fails, times out, or returns invalid structured output, the system retries it up to three times using the same prompt, parameters, and schema. Persistent failures are recorded in the execution log.

Metrics For the binary delegation task, we report accuracy, balanced accuracy, class-specific precision and recall, and macro-F1. For the ordinal discretion task, we report exact-match accuracy, within-one accuracy (W_1), mean absolute error, linear weighted kappa, and quadratic weighted kappa. Because weighted kappa is sensitive to marginal class imbalance on smaller samples, W_1 serves as our primary operational indicator of alignment for expert screening.

4.5 Results and Diagnostics

Binary delegation Table 3 reports binary delegation screening results on the full available delegation benchmark ($N = 169$). The benchmark is severely imbalanced (161 positive cases and only 8 negative cases). We compare the production Delegation Gate against an earlier broad-screening baseline (*High-Recall Screen*). The High-Recall Screen relied on broad semantic matching for any grant of power. The Delegation Gate revised the prompt to include explicit negative exclusions: routine administrative housekeeping, technical statutory amendments, and advisory commissions lacking binding rulemaking or enforcement authority.

Table 3. Binary delegation stage screening results ($N = 169$).

Metric	High-Recall Screen	Delegation Gate
Overall Accuracy	94.67%	91.72%
Balanced Accuracy	73.45%	89.71%
Positive Recall (Delegation)	96.89%	91.93%
Negative Recall	50.00%	87.50%
Positive Precision	97.50%	99.33%
Negative Precision	44.44%	35.00%
Macro-F1	72.13%	72.74%
TP / FN / TN / FP	156 / 5 / 4 / 4	148 / 13 / 7 / 1

Although a naive majority-class baseline predicting delegation for all cases achieves 95.27% raw accuracy, it yields only 50.00% balanced accuracy and fails entirely on non-delegation statutes (0.00% negative recall). In contrast, while the High-Recall Screen yields slightly higher raw accuracy due to class imbalance, the conservative Delegation Gate detects seven of the eight rare negative cases (87.50% negative recall), achieving 89.71% balanced accuracy and 72.74% macro-F1.

Diagnosing disagreement The law PL83-577 illustrates why source discipline and intermediate evidence matter. The benchmark codes it as non-delegation (rank 0). An exploratory staged run instead identified Securities and Exchange Commission rulemaking and exemption authority and returned delegation. This disagreement could reflect an overbroad coding rule, a source-universe difference,

Table 4. Cross-model evaluation results across ranking strategies and unconstrained baseline.

Model	Strategy	N	Exact	W1	MAE	Linear κ_w	Quad κ_w
Gemini 3.5	One-Shot Baseline (Unconstrained)	140	28.6%	62.1%	1.250	0.051	0.042
	Sequential Ordinal Cascade	117	29.9%	75.2%	1.077	0.104	0.094
	Direct Multiclass Prediction	117	41.9%	76.9%	0.915	0.207	0.157
	Two-Band Classification	117	29.9%	65.8%	1.188	0.044	0.010
Qwen 3	Sequential Ordinal Cascade	129	25.6%	65.9%	1.225	0.062	0.059
	Direct Multiclass Prediction	129	32.6%	64.3%	1.171	0.105	0.093
	Two-Band Classification	129	17.8%	48.8%	1.620	0.030	0.010
DeepSeek	Sequential Ordinal Cascade	129	20.9%	57.4%	1.434	0.022	0.010
	Direct Multiclass Prediction	129	29.5%	71.3%	1.140	0.055	0.014
	Two-Band Classification	129	8.5%	41.9%	1.837	-0.014	-0.020
Kimi K2.5	Sequential Ordinal Cascade	60	30.0%	63.3%	1.317	0.133	0.094
	Direct Multiclass Prediction	60	21.7%	60.0%	1.433	0.076	0.067
	Two-Band Classification	60	15.0%	45.0%	1.700	-0.008	-0.003
GPT-OSS	Sequential Ordinal Cascade	85	20.0%	55.3%	1.482	0.056	0.067
	Direct Multiclass Prediction	85	32.9%	69.4%	1.129	0.106	0.072
	Two-Band Classification	85	9.4%	37.6%	1.871	0.007	0.021

Note: N reports completed evaluations per configuration; varying sample sizes across rows reflect provider rate limits during evaluation runs. The unconstrained One-Shot Baseline is evaluated on Gemini 3.5.

or a mismatch between historical summary coding and full statutory detail. The trace does not by itself determine which label is correct; it makes the source of the disagreement visible for expert adjudication.

Discretion rank and model comparison Table 4 reports the cross-model evaluation of the three ranking strategies alongside an unconstrained One-Shot baseline on the expert-coded delegated-law benchmark ($N = 161$ total available delegated statutes, with completed sample sizes reported per configuration).

For Gemini 3.5, Direct Multiclass Rank Prediction reports the strongest aggregate results: exact-match accuracy increases from 28.6% for the One-Shot Baseline to 41.9%, within-one accuracy rises from 62.1% to 76.9%, and mean absolute error falls from 1.250 to 0.915. Here, the One-Shot Baseline serves as an illustrative comparison for Gemini 3.5, demonstrating workflow gains rather than a universal empirical result across all models.

Within-one accuracy exceeds exact-match accuracy across all configurations (reaching 76.9% for Gemini 3.5 and 71.3% for DeepSeek), confirming that most errors are adjacent. Because legal codebooks require nuanced boundary judgments on statutory constraints, adjacent agreement indicates strong substantive proximity. Rather than replacing human expertise, the workflow functions as a triage and auditing instrument: it reliably screens non-delegation laws (89.7% balanced accuracy) and surfaces structured evidence rationales, enabling scholars to audit boundary cases without manually re-coding entire statutes.

5 Discussion, Limitations, and Conclusion

Reproducibility. Replication requires preserving the source set, stable law identifiers, workflow definition, prompts, typed outputs, model settings, execution trace, benchmark comparison, case-level adjudications, and exported dataset.

Because the workflow, rather than a single prompt, is the measurement instrument, these materials must remain linked to the version used to produce each coded output.

Limitations. Three limitations remain. First, the evaluation relies on legislative summaries rather than full statutory text, preserving comparability with the expert benchmark while leaving full-text validation for future work. Second, the benchmark reflects U.S. financial regulation, preserving within-domain construct validity while leaving cross-domain external validity for subsequent studies. Third, LLM classifications remain sensitive to model choice, prompt wording, and provider updates.

Future Work. Future work will evaluate held-out laws, measure component-level authority and constraint accuracy, test additional policy domains, extend one-shot baselines across model backends, and use independent expert recoding to establish human intercoder reliability.

Conclusions. This paper addressed the challenge of recovering delegated authority and administrative discretion from legal text. Rather than relying on black-box classifiers or opaque prompts, we introduced a theory-guided LLM workflow where delegation, authority, and constraints are modeled as distinct, interdependent components with deterministic theoretical mapping. Human experts govern the rules, validation, and adjudication. The resulting system makes latent institutional measurement explicit, structured, inspectable, and reproducible.

References

1. Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D., Aletras, N.: LexGLUE: A benchmark dataset for legal language understanding in english. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics. pp. 4310–4330. Association for Computational Linguistics (2022). https://doi.org/10.18653/v1/2022.acl-long.297
2. Cormack, G.V., Grossman, M.R.: Evaluation of machine-learning protocols for technology-assisted review in electronic discovery. In: Proceedings of the 37th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 153–162. ACM (2014). https://doi.org/10.1145/2600428.2609601
3. Dai, S.C., Xiong, A., Ku, L.W.: LLM-in-the-loop: Leveraging large language model for thematic analysis. In: Findings of the Association for Computational Linguistics: EMNLP 2023. pp. 9993–10001. Association for Computational Linguistics (2023). https://doi.org/10.18653/v1/2023.findings-emnlp.669
4. Epstein, D., O'Halloran, S.: Administrative procedures, information, and agency discretion. American Journal of Political Science **38**(3), 697–722 (1994)
5. Epstein, D., O'Halloran, S.: Delegating Powers: A Transaction Cost Politics Approach to Policy Making under Separate Powers. Cambridge University Press, Cambridge (1999)
6. Gilardi, F., Alizadeh, M., Kubli, M.: Chatgpt outperforms crowd workers for text-annotation tasks. Proceedings of the National Academy of Sciences **120**(30), e2305016120 (2023). https://doi.org/10.1073/pnas.2305016120

7. Grimmer, J., Stewart, B.M.: Text as data: The promise and pitfalls of automatic content analysis methods for political texts. Political Analysis **21**(3), 267–297 (2013). https://doi.org/10.1093/pan/mps028
8. Groll, T., O'Halloran, S., McAllister, G.: Delegation and the regulation of u.s. financial markets. European Journal of Political Economy **70**, 102058 (2021). https://doi.org/10.1016/j.ejpoleco.2021.102058
9. Guha, N., Nyarko, J., Ho, D.E., Ré, C., Chilton, A., et al.: LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In: Advances in Neural Information Processing Systems. vol. 36 (2023), arXiv:2308.11462
10. O'Halloran, S., et al.: Big data and the regulation of banking and financial services. Banking & Financial Services Policy Report **34**(12), 1–10 (2015)
11. O'Halloran, S., et al.: Big data and the regulation of financial markets. In: Proceedings of the 2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining. pp. 1118–1124. ACM (2015)
12. O'Halloran, S., et al.: Computational data sciences and the regulation of banking and financial services. In: Alhajj, R. (ed.) Social Network Analysis Lecture Notes. Springer, New York (2016)
13. O'Halloran, S., et al.: Data science and political economy: Application to financial regulatory structure. RSF: The Russell Sage Foundation Journal of the Social Sciences **2**(7) (2016)
14. O'Halloran, S., et al.: Big data and graph theoretic models: Simulating the impact of collateralization on financial system. In: Proceedings of the 2017 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining. pp. 1056–1064. ACM (2017)
15. Ratner, A., Bach, S.H., Ehrenberg, H., Fries, J., Wu, S., Ré, C.: Snorkel: Rapid training data creation with weak supervision. Proceedings of the VLDB Endowment **11**(3), 269–282 (2017). https://doi.org/10.14778/3157794.3157797
16. Thalken, R., Stiglitz, E., Mimno, D., Wilkens, M.: Modeling legal reasoning: LM annotation at the edge of human agreement. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 9252–9265. Association for Computational Linguistics (2023). https://doi.org/10.18653/v1/2023.emnlp-main.575
17. Wu, T., Jiang, E., Donsbach, A., Gray, J., Molina, A., Terry, M., Cai, C.J.: Promptchainer: Chaining large language model prompts through visual programming. In: CHI Conference on Human Factors in Computing Systems Extended Abstracts. ACM (2022). https://doi.org/10.1145/3491101.3519729
18. Wu, T., Terry, M., Cai, C.J.: Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts. In: Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems. ACM (2022). https://doi.org/10.1145/3491102.3517582